

Chapitre 8 : Traits Impératifs en OCaml

I Les Références

Une **référence** en OCaml est une cellule mémoire allouée dynamiquement contenant une valeur modifiable d'un type donné. Conceptuellement identique à un pointeur initialisé en C ou à une référence d'objet mutable en Java, elle permet d'introduire des variables d'état locales ou globales.

A Typage et Opérations Fondamentales

Le constructeur de type `ref` est polymorphe. Si t est un type OCaml, alors $t\ \text{ref}$ est le type d'une référence pointant vers une valeur de type t .

Les trois opérations fondamentales disposent des signatures suivantes :

- **Allocation** : `ref : 'a -> 'a ref` → Alloue une cellule en mémoire et l'initialise.
- **Lecture (Déréférencement)** : `! : 'a ref -> 'a` → Accède à la valeur courante stockée.
- **Écriture (Affectation)** : `(:=) : 'a ref -> 'a -> unit` → Modifie le contenu de la cellule.

💡 **Exemple** : Exemple d'utilisation basique :

```
1 let r = ref 42          (* r : int ref, allocation initiale *)
2 let v = !r              (* v = 42, lecture du contenu *)
3 let () = r := 100       (* Modifie le contenu de r. Renvoie () *)
```

B Aliasing : partage de références

L'affectation d'une référence à une autre variable copie l'adresse mémoire (le pointeur) et non la valeur sous-jacente. Il y a alors création d'un **alias**.

💡 **Exemple** : Mise en évidence du partage en mémoire :

```
1 let r1 = ref 5 in
2 let r2 = r1 in          (* r2 pointe vers la meme cellule memoire que r1 *)
3 r2 := 12;
4 !r1                     (* Evalue a 12, car le contenu lie a r1 a ete modifie via r2 *)
   *
```

II Structures de contrôle : les boucles

OCaml intègre des boucles impératives pour exécuter des blocs de code produisant des effets de bord (expressions de type `unit`). Les instructions d'une boucle sont séparées par le séquenceur `;`.

A La boucle bornée (`for`)

La boucle `for` utilise un compteur entier incrémenté (`to`) ou décrémenté (`downto`) de 1 à chaque itération. L'indice de la boucle est une valeur immuable locale au corps de la boucle.

```
1 (* Ordre croissant *)
2 for i = t1 to t2 do
3   expr
4 done
5
6 (* Ordre décroissant *)
7 for i = t1 downto t2 do
```

```

8   expr
9   done

```

B La boucle non bornée (while)

La boucle `while` évalue son expression conditionnelle (qui doit être de type `bool`) avant chaque itération. Le corps est exécuté tant que la condition vaut `true`.

```

1   while condition do
2     expr
3   done

```

💡 **Exemple** : Calcul itératif de la factorielle à l'aide d'une référence et d'une boucle :

```

1   let facto n =
2     let res = ref 1 in
3     let i = ref 1 in
4     while !i <= n do
5       res := !res * !i;
6       i := !i + 1
7     done;
8     !res

```

III Enregistrements à champs modifiables

Par défaut, les champs d'un enregistrement (`record`) sont immuables. Il est possible de rendre certains champs mutables individuellement lors de la définition du type à l'aide du mot-clé `mutable`.

A Définition et Syntaxe d'Affectation

```

1   type point_mutable = {
2     mutable x : float;
3     mutable y : float;
4     label : string      (* Ce champ reste immuable *)
5   }

```

L'accès à un champ se fait via la notation pointée (`p.x`). La modification d'un champ mutable utilise l'opérateur d'affectation destructif `<-`.

💡 **Exemple** : Modification en place d'un enregistrement :

```

1   let p = { x = 0.0; y = 1.0; label = "Origine" }
2   let () = p.x <- 5.5      (* Valide, le champ x est mutable *)
3   (* p.label <- "A" *)    (* Erreur de typage : le champ label est immuable *)

```

❗ **Remarque** : En réalité, le type primitif `'a ref` d'OCaml est défini nativement sous la forme d'un enregistrement prédéfini à un seul champ mutable : `type 'a ref = { mutable contents : 'a }`. L'écriture `r := v` équivaut sémantiquement à `r.contents <- v`.

IV Les tableaux (Arrays)

Le type `'a array` représente des structures de données contiguës en mémoire, de taille fixe, dont les éléments sont indexés de 0 à $n - 1$ et modifiables en place.

A Création et manipulation

- **Syntaxe littérale** : `[| e0; e1; e2 |]`
- **Allocation par fonction** : `Array.make : int -> 'a -> 'a array` → Crée un tableau d'une taille donnée dont chaque case est initialisée avec la *même* valeur fournie.
- **Accès en lecture** : `t.(i)` (équivalent à `Array.get t i`).
- **Accès en écriture** : `t.(i) <- v` (équivalent à `Array.set t i v`).

B Piège : partage de mémoire (*aliasing* de sous-structures)

Lors de l'utilisation de `Array.make` pour allouer des structures multidimensionnelles (tableaux de tableaux), OCaml duplique la référence de l'argument d'initialisation et non sa valeur.

💡 **Exemple** : Analyse d'un comportement problématique :

```
1 (* Tentative de creation d'une matrice 3x2 initialisee a 0 *)
2 let m = Array.make 3 (Array.make 2 0);;
3 (* m : int array array = [| [0; 0]; [0; 0]; [0; 0] |] *)
4
5 m.(0).(0) <- 99;;
6 (* Problème : Toutes les lignes de la matrice ont ete modifiees ! *)
7 (* m est en realite : [| [99; 0]; [99; 0]; [99; 0] |] *)
```

Explication sémantique : L'expression interne `Array.make 2 0` est évaluée **une seule fois**, générant un unique sous-tableau en mémoire. La fonction externe `Array.make 3` crée ensuite un tableau principal contenant trois pointeurs dirigés vers cet unique et même sous-tableau.

Solution : Pour instancier correctement des structures de données mutables imbriquées sans partage de pointeurs, il faut utiliser `Array.make_matrix` ou la fonction d'initialisation par fonction `Array.init` :

```
1 (* Allocation correcte sans aliasing : chaque ligne est un bloc memoire distinct *)
2 let m_correct = Array.init 3 (fun _ -> Array.make 2 0)
```

Contents

I	Les Références	1
A	Typage et Opérations Fondamentales	1
B	Aliasing : partage de références	1
II	Structures de contrôle : les boucles	1
A	La boucle bornée (<code>for</code>)	1
B	La boucle non bornée (<code>while</code>)	2
III	Enregistrements à champs modifiables	2
A	Définition et Syntaxe d'Affectation	2
IV	Les tableaux (Arrays)	2
A	Création et manipulation	3
B	Piège : partage de mémoire (<i>aliasing</i> de sous-structures)	3